LAPORAN

WORKSHOP SISTEM INFORMASI

WEB SERVER

(TIF330805)

SEMESTER III



BKPM Acara 9 Oop Mvc Lanjutan: Object, Dependency, Getter Setter

NAMA:

AZIRA MUTIA GANI

NIM:

E41250290

PROGRAM STUDI DILUAR KAMPUS UTAMA (PSDKU)

TEKNIK INFORMATIKA KAB. NGANJUK

JURUSAN TEKNOLOGI INFORMASI

POLITEKNIK NEGERI JEMBER

TAHUN 2026

BAB 1. PENDAHULUAN

1.1 Tujuan Praktikum

1. Memahami Object Composition.
2. Memahami Dependency Injection.
3. Menggunakan Getter dan Setter.
4. Menerapkan validasi pada Setter.
5. Melakukan refactoring CRUD mahasiswa menggunakan konsep OOP.

Dasar Teori

1. Object Composition adalah membuat sebuah class memiliki object dari class lain.
   Contohnya:
   Controller → memiliki Repository → memiliki Database
2. Dependency Injection adalah memberikan object yang dibutuhkan sebuah class melalui
   constructor.
3. Getter digunakan untuk mengambil nilai atribut.
4. Setter digunakan untuk mengubah nilai atribut dan dapat digunakan untuk melakukan
   validasi.

BAB 2. LANDASAN TEORI

2.1 PHP Data Objects (PDO)

PHP Data Objects (PDO) merupakan fitur PHP yang digunakan untuk mengakses dan mengelola basis data melalui antarmuka yang konsisten. PDO mendukung berbagai jenis basis data, termasuk MySQL. Dengan PDO, pengembang dapat menjalankan perintah SQL untuk mengambil, menambahkan, memperbarui, dan menghapus data. Penggunaan PDO membantu membuat kode program lebih terstruktur serta mempermudah pengelolaan koneksi basis data.

1.1 Database

Database atau basis data merupakan kumpulan data yang disimpan secara terorganisasi sehingga dapat diakses dan dikelola dengan mudah. Dalam Sistem Informasi Akademik, database digunakan untuk menyimpan informasi seperti data mahasiswa, program studi, dan mata kuliah. Pengelolaan data melalui database membantu menjaga konsistensi informasi serta mempermudah proses pencarian dan perubahan data.

1.2 CRUD (Create, Read, Update, Delete)

CRUD merupakan empat operasi dasar yang digunakan untuk mengelola data dalam aplikasi. Create digunakan untuk menambahkan data baru, Read untuk menampilkan atau membaca data, Update untuk memperbarui data yang telah tersimpan, dan Delete untuk menghapus data. Pada aplikasi SI-Akademik, operasi CRUD dapat diterapkan untuk mengelola data mahasiswa, program studi, dan mata kuliah sehingga proses pengelolaan data dapat dilakukan melalui antarmuka aplikasi.

1.3 Prepared Statement

Prepared statement merupakan metode untuk menyiapkan perintah SQL dengan parameter yang nilainya diberikan secara terpisah. Dalam PDO, prepared statement dapat digunakan melalui metode `prepare()` dan dijalankan menggunakan `execute()`. Metode ini membantu memisahkan struktur perintah SQL dari masukan pengguna sehingga dapat mengurangi risiko SQL injection apabila digunakan dengan benar. Prepared statement juga membuat proses pengiriman nilai ke dalam query menjadi lebih terstruktur.

1.4 Model dalam Pengelolaan Data

Model merupakan bagian dari pola arsitektur aplikasi yang bertanggung jawab mengelola data dan berinteraksi dengan basis data. Dalam aplikasi SI-Akademik, model dapat digunakan untuk menangani proses pengambilan, penambahan, perubahan, dan penghapusan data mahasiswa atau program studi. Pemisahan pengelolaan data ke dalam model membantu mengurangi penulisan query secara berulang serta membuat kode program lebih mudah dipelihara.

1.5 Implementasi CRUD pada Sistem Informasi Akademik

Implementasi CRUD pada Sistem Informasi Akademik memungkinkan pengguna mengelola data melalui halaman aplikasi. PHP dan PDO digunakan untuk menghubungkan aplikasi dengan basis data, sedangkan perintah SQL digunakan untuk menjalankan operasi terhadap data. Dengan menerapkan operasi CRUD dan prepared statement, pengelolaan data mahasiswa, program studi, dan mata kuliah dapat dilakukan secara lebih terstruktur serta memperhatikan keamanan pemrosesan masukan pengguna.

BAB 3. PELAKSANAAN PRAKTIKUM

3.1 Hasil Praktik

A. Studi Kasus

Pada praktikum ini dilakukan refactoring aplikasi mahasiswa dengan menerapkan Object Composition, Dependency Injection, serta Getter dan Setter.

Database dikelola oleh class Database, kemudian object tersebut diberikan kepada MahasiswaRepository. Repository selanjutnya diberikan kepada MahasiswaController.

Struktur hubungan object: Database - MahasiswaRepository - MahasiswaController - View

Class Mahasiswa menggunakan atribut private dan menyediakan Getter serta Setter. Setter juga digunakan untuk melakukan validasi NIM dan nama.

1. Class Mahasiswa

`app/Models/Mahasiswa.php`

Semua atribut dibuat private.

Getter digunakan untuk mengambil data dan Setter digunakan untuk mengubah data sekaligus melakukan validasi.

```php
<?php

class Mahasiswa
{
    private int $id;
    private string $nim;
    private string $nama;
    private string $prodi;
    private string $status;
    private ?int $dosen_id;

    public function __construct(
        string $nim = '',
        string $nama = '',
        string $prodi = '',
        string $status = 'aktif',
        ?int $dosen_id = null
    ) {
```

```
        $this->setNim($nim);
        $this->setNama($nama);
        $this->setProdi($prodi);
        $this->setStatus($status);
        $this->setDosenId($dosen_id);
    }

    public function getId(): int
    {
        return $this->id;
    }

    public function getNim(): string
    {
        return $this->nim;
    }

    public function getNama(): string
    {
        return $this->nama;
    }

    public function getProdi(): string
    {
        return $this->prodi;
    }

    public function getStatus(): string
    {
        return $this->status;
    }

    public function getDosenId(): ?int
    {
        return $this->dosen_id;
    }

    public function setId(int $id): void
    {
        $this->id = $id;
    }

    public function setNim(string $nim): void
    {
        if ($nim === '' || !ctype_digit($nim)) {
            throw new InvalidArgumentException(
                'NIM harus berupa angka.'
```

```
            );
        }

        $this->nim = $nim;
    }

    public function setNama(string $nama): void
    {
        $nama = trim($nama);

        if ($nama === '') {
            throw new InvalidArgumentException(
                'Nama tidak boleh kosong.'
                );
        }

        $this->nama = $nama;
    }

    public function setProdi(string $prodi): void
    {
        $prodi = trim($prodi);

        if ($prodi === '') {
            throw new InvalidArgumentException(
                'Prodi tidak boleh kosong.
                ');
        }

        $this->prodi = $prodi;
    }

    public function setStatus(string $status): void
    {
        if (!in_array($status, ['aktif', 'cuti', 'lulus'])) {
            throw new InvalidArgumentException(
                'Status tidak valid.'
            );
        }

        $this->status = $status;
    }

    public function setDosenId(?int $dosen_id): void
    {
        $this->dosen_id = $dosen_id;
    }
```

```
}
```

2. Mahasiswa Repository

`app/Repositories/MahasiswaRepository.php`

Repository menerima object Database melalui constructor.

```php
<?php

require_once __DIR__ . '/../Core/Database.php';
require_once __DIR__ . '/../Models/Mahasiswa.php';

class MahasiswaRepository
{
    private PDO $db;

    public function __construct(PDO $db)
    {
        $this->db = $db;
    }

    public function all(): array
    {
        $stmt = $this->db->query(
            "SELECT * FROM mahasiswa ORDER BY id DESC"
        );

        return $stmt->fetchAll();
    }

    public function find(int $id): ?array
    {
        $stmt = $this->db->prepare(
            "SELECT * FROM mahasiswa WHERE id = :id"
        );

        $stmt->execute([
            'id' => $id
        ]);

        $data = $stmt->fetch();

        return $data ?: null;
    }

    public function create(Mahasiswa $mahasiswa): bool
    {
        $stmt = $this->db->prepare("
            INSERT INTO mahasiswa
```

```
            (nim, nama, prodi, status, dosen_id)
            VALUES
            (:nim, :nama, :prodi, :status, :dosen_id)
        ");

        return $stmt->execute([
            'nim' => $mahasiswa->getNim(),
            'nama' => $mahasiswa->getNama(),
            'prodi' => $mahasiswa->getProdi(),
            'status' => $mahasiswa->getStatus(),
            'dosen_id' => $mahasiswa->getDosenId()
        ]);
    }

    public function update(int $id, Mahasiswa $mahasiswa): bool
    {
        $stmt = $this->db->prepare("
            UPDATE mahasiswa SET
                nim = :nim,
                nama = :nama,
                prodi = :prodi,
                status = :status,
                dosen_id = :dosen_id
            WHERE id = :id
        ");

        return $stmt->execute([
            'id' => $id,
            'nim' => $mahasiswa->getNim(),
            'nama' => $mahasiswa->getNama(),
            'prodi' => $mahasiswa->getProdi(),
            'status' => $mahasiswa->getStatus(),
            'dosen_id' => $mahasiswa->getDosenId()
        ]);
    }

    public function delete(int $id): bool
    {
        $stmt = $this->db->prepare(
            "DELETE FROM mahasiswa WHERE id = :id"
        );

        return $stmt->execute([
            'id' => $id
        ]);
    }
}
```

3. Mahasiswa Controller

`app/Controllers/MahasiswaController.php`

Perhatikan bahwa Controller tidak membuat koneksi database langsung. Repository diberikan melalui constructor.

```php
<?php

require_once __DIR__ . '/../Models/Mahasiswa.php';
require_once __DIR__ . '/../Repositories/MahasiswaRepository.php';

class MahasiswaController
{
    private MahasiswaRepository $repo;

    public function __construct(MahasiswaRepository $repo)
    {
        $this->repo = $repo;
    }

    public function index(): void
    {
        $mahasiswa = $this->repo->all();

        $content = __DIR__ . '/../Views/mahasiswa/index.php';

        require __DIR__ . '/../Views/layouts/main.php';
    }

    public function create(): void
    {
        $content = __DIR__ . '/../Views/mahasiswa/create.php';

        require __DIR__ . '/../Views/layouts/main.php';
    }

    public function store(): void
    {
        try {

            $mahasiswa = new Mahasiswa(
                trim($_POST['nim']),
                trim($_POST['nama']),
                trim($_POST['prodi']),
                $_POST['status'],
                !empty($_POST['dosen_id'])
```

```
                ? (int) $_POST['dosen_id']
                : null
        );

        $this->repo->create($mahasiswa);

        header(
            'Location: ' . BASE_URL . '/mahasiswa'
        );

        exit;

    } catch (InvalidArgumentException $e) {

        echo $e->getMessage();
    }
}

public function edit(int $id): void
{
    $mahasiswa = $this->repo->find($id);

    if (!$mahasiswa) {
        http_response_code(404);
        echo 'Data mahasiswa tidak ditemukan.';
        return;
    }

    $content = __DIR__ . '/../Views/mahasiswa/edit.php';

    require __DIR__ . '/../Views/layouts/main.php';
}

public function update(int $id): void
{
    try {

        $mahasiswa = new Mahasiswa(
            trim($_POST['nim']),
            trim($_POST['nama']),
            trim($_POST['prodi']),
            $_POST['status'],
            !empty($_POST['dosen_id'])
                ? (int) $_POST['dosen_id']
                : null
        );
```

```
            $this->repo->update($id, $mahasiswa);

            header(
                'Location: ' . BASE_URL . '/mahasiswa'
            );

            exit;

        } catch (InvalidArgumentException $e) {

            echo $e->getMessage();
        }
    }

    public function destroy(int $id): void
    {
        $this->repo->delete($id);

        header(
            'Location: ' . BASE_URL . '/mahasiswa'
        );

        exit;
    }
}
```

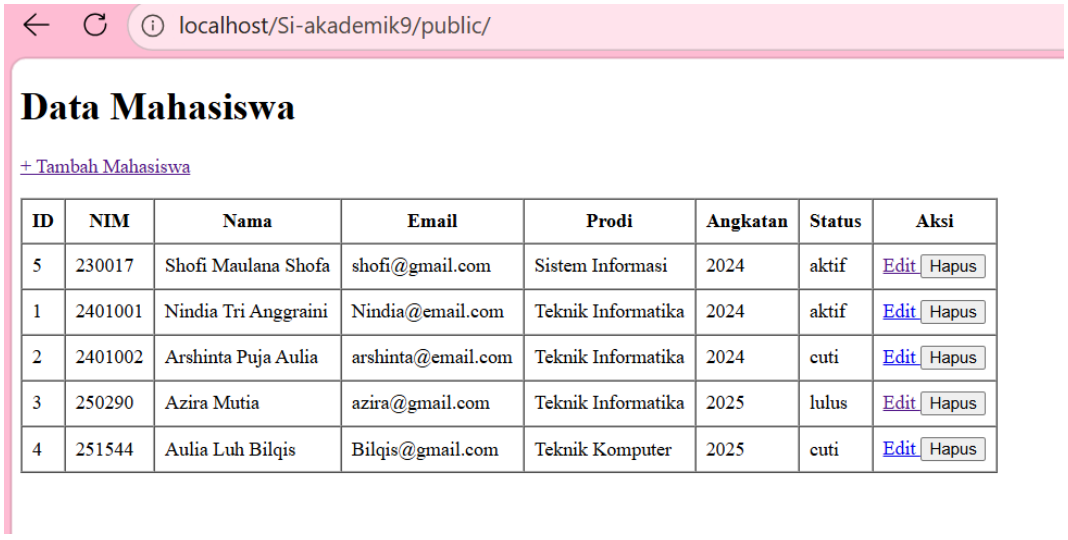
Hasil Studi Kasus

1. Uji Read

http://localhost/BKPM/si-akademik9/public/mahasiswa

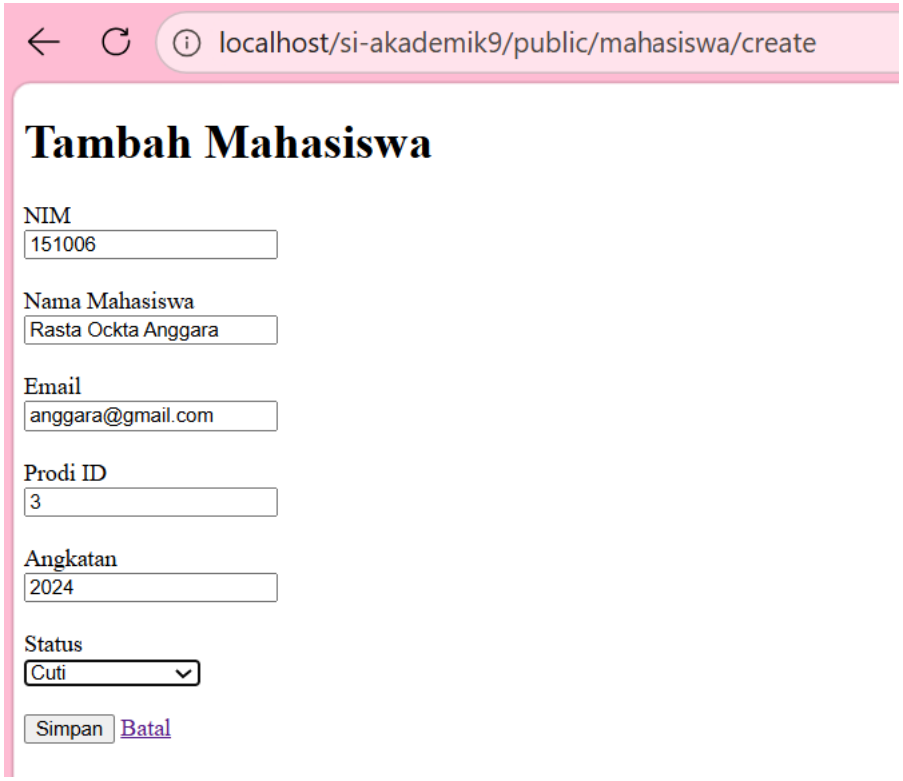
Data mahasiswa tampil pada halaman.

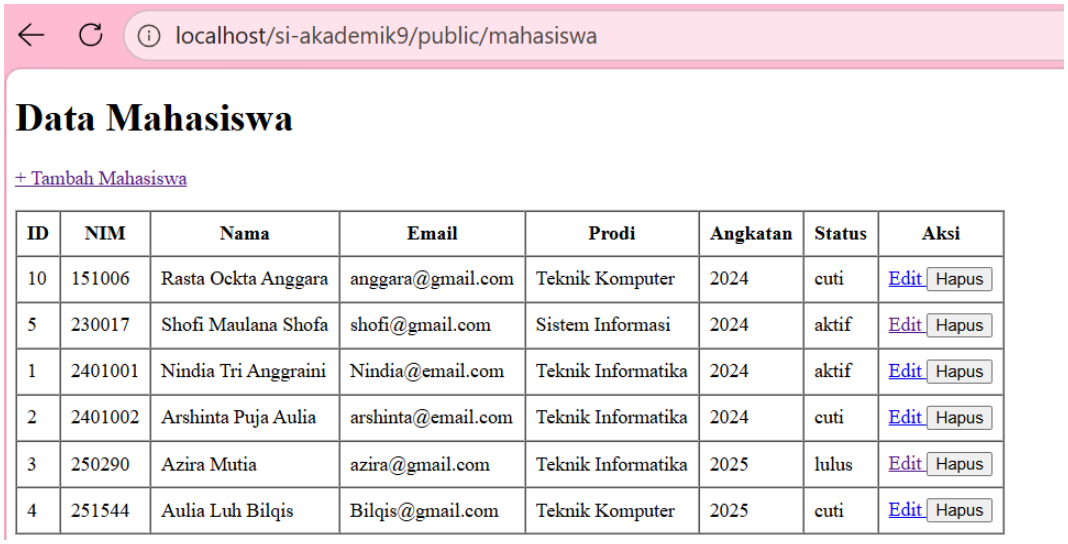
Hasil: berhasil menampilkan data mahasiswa.



2. Uji Create

   Masukkan:



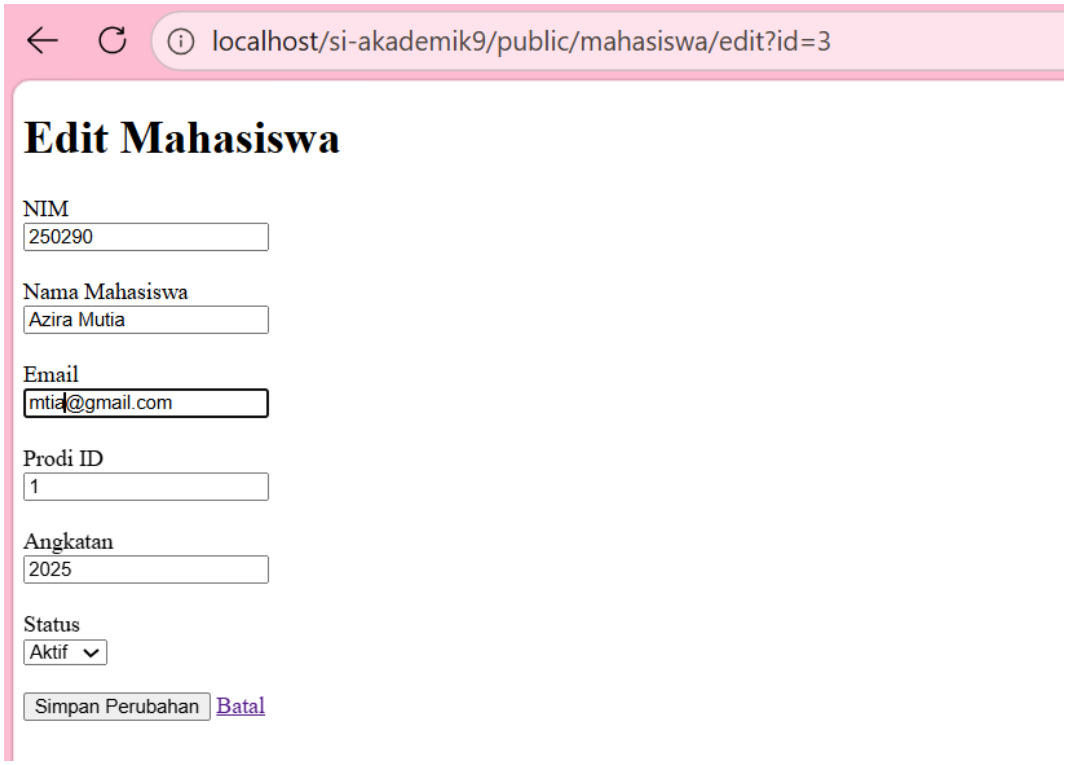


ID	NIM	Nama	Email	Prodi	Angkatan	Status	Aksi
10	151006	Rasta Ockta Anggara	anggara@gmail.com	Teknik Komputer	2024	cuti	Edit Hapus
5	230017	Shofi Maulana Shofa	shofi@gmail.com	Sistem Informasi	2024	aktif	Edit Hapus
1	2401001	Nindia Tri Anggraini	Nindia@email.com	Teknik Informatika	2024	aktif	Edit Hapus
2	2401002	Arshinta Puja Aulia	arshinta@email.com	Teknik Informatika	2024	cuti	Edit Hapus
3	250290	Azira Mutia	azira@gmail.com	Teknik Informatika	2025	lulus	Edit Hapus
4	251544	Aulia Luh Bilqis	Bilqis@gmail.com	Teknik Komputer	2025	cuti	Edit Hapus

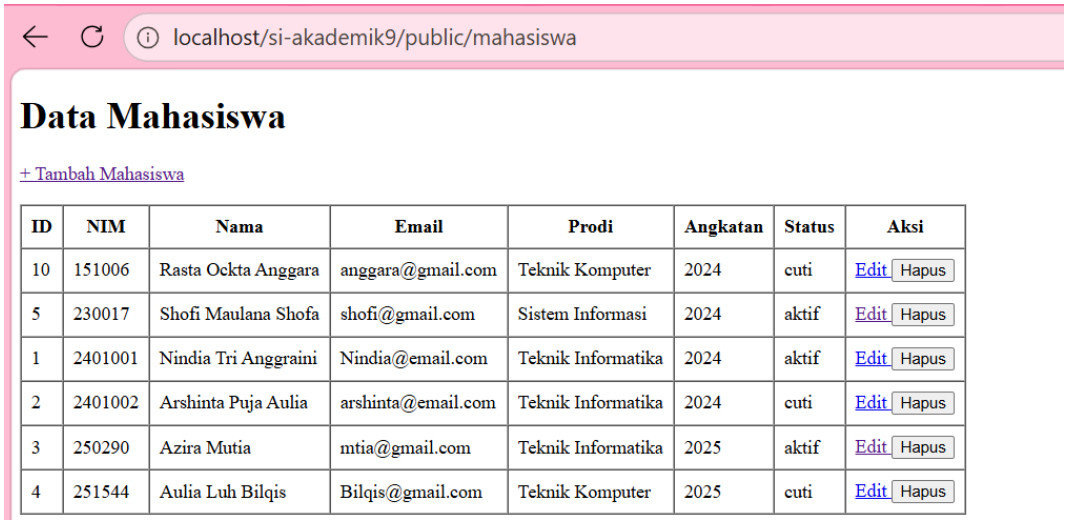
   Hasil: data berhasil masuk ke database.

3. Uji Update

   Klik Edit, kemudian ubah nama:



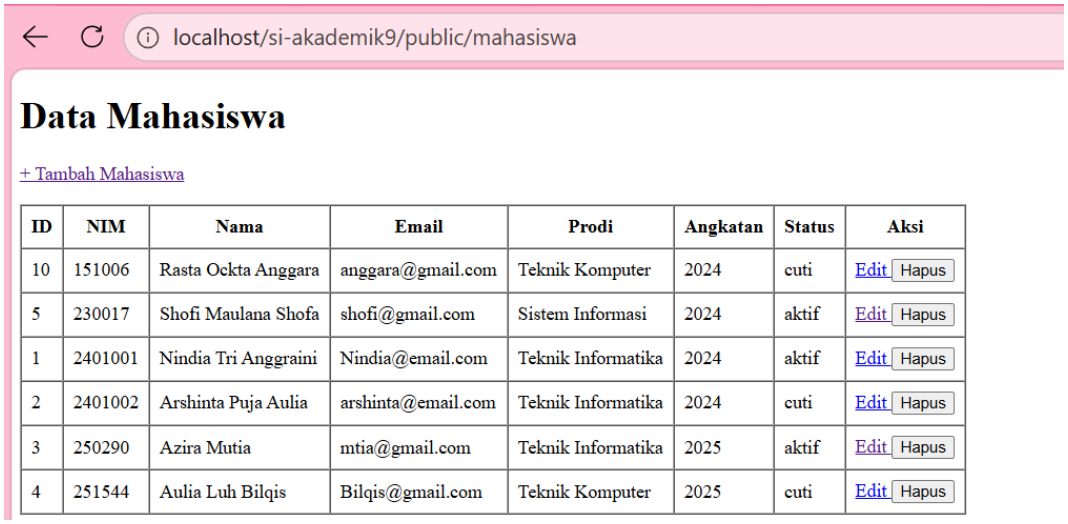
Hasil: data berhasil diperbarui.



4. Uji Delete

   Klik Hapus.

   Konfirmasi penghapusan.



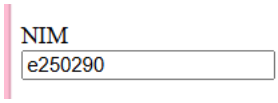


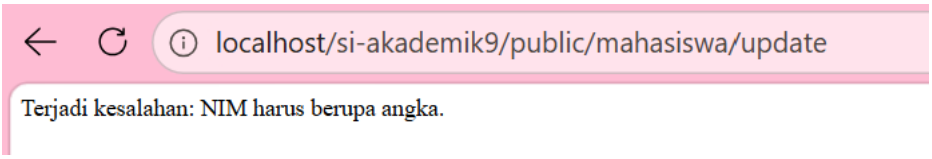


Hasil: data berhasil dihapus.

5. Uji Validasi Setter

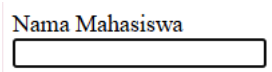
Coba memasukkan NIM:





Setter setNim() akan menolak data karena NIM harus berupa angka.

Coba kosongkan nama.





Setter setNama() akan menolak data karena nama tidak boleh kosong.

Analisis :

Pada Acara 9, kode CRUD diperbaiki dengan konsep OOP. Controller tidak lagi membuat koneksi database secara langsung. Dependency diberikan melalui constructor sehingga kode lebih terstruktur. Getter dan Setter digunakan untuk mengatur akses atribut dan melakukan validasi data.

BAB 4. PENUTUP

5.1 Kesimpulan

Praktikum Acara 9 berhasil menerapkan Object Composition, Dependency Injection, Getter, dan Setter pada aplikasi MVC. Proses CRUD tetap dapat berjalan setelah dilakukan refactoring, serta validasi sederhana berhasil diterapkan pada data mahasiswa.